PROBLEM SESSION 3: Assembly Language

Member:

- Naphat Utabuawong 67070501015
- Thanatip Nitinantakul 67070501023
- Benjamin Garforth 67070501031

Source code:

```
Data
        DCD     21, 10, 8, 1, 5, 7, 11, 6, 8

MAIN
        LDR     R0, =Data
        MOV     R1, #0
        MOV     R2, #8
        BL      QUICKSORT

STOP
        B       STOP


PARTITION
        STMFD   SP!, {R4-R8, LR}

        LSL     R3, R2, #2
        ADD     R3, R0, R3
        LDR     R4, [R3]

        SUB     R5, R1, #1
        MOV     R6, R1

PART_LOOP
        CMP     R6, R2
        BGE     PART_END

        LSL     R7, R6, #2
        ADD     R7, R7, R0
        LDR     R8, [R7]

        CMP     R8, R4
        BGT     PART_SKIP
```

Source code (continued):

```
        ADD     R5, R5, #1

        LSL     R7, R5, #2
        ADD     R7, R7, R0

        LSL     R8, R6, #2
        ADD     R8, R8, R0

        LDR     R3, [R7]
        LDR     R12, [R8]
        STR     R12, [R7]
        STR     R3, [R8]

PART_SKIP
        ADD     R6, R6, #1
        B       PART_LOOP

PART_END
        ADD     R5, R5, #1

        LSL     R7, R5, #2
        ADD     R7, R7, R0

        LSL     R8, R2, #2
        ADD     R8, R8, R0

        LDR     R3, [R7]
        LDR     R12, [R8]
        STR     R12, [R7]
        STR     R3, [R8]

        MOV     R0, R5
        LDMFD   SP!, {R4-R8, PC}


QUICKSORT
        STMFD   SP!, {R4-R6, LR}

        CMP     R1, R2
        BGE     QS_END

        MOV     R4, R0
        MOV     R5, R1
        MOV     R6, R2
```

Source code (continued):

```
        BL      PARTITION

        MOV     R3, R0

        MOV     R0, R4
        MOV     R1, R5
        SUB     R2, R3, #1
        BL      QUICKSORT

        MOV     R0, R4
        ADD     R1, R3, #1
        MOV     R2, R6
        BL      QUICKSORT

QS_END
        LDMFD   SP!, {R4-R6, PC}

        END
```

Explanation of source code:

Assembly Code	Explanation
Data DCD 21,10,8,1,5,7,11,6,8	ค่าที่ตั้งไว้เพื่อนำไปเข้า quick sort algorithm

MAIN
LDR R0, =Data
MOV R1, #0
MOV R2, #8
BL QUICKSORT
	- โหลดที่อยู่ของอาเรย์ Data ไปที่ register R0
	- กำหนดค่า low = 0 (ตำแหน่งเริ่มต้นของอาเรย์) ไปที่ register R1
	- กำหนดค่า high = 8 (ตำแหน่งสุดท้ายของอาเรย์) ไปที่ register R2
	- เรียกฟังก์ชัน QUICKSORT เพื่อทำการเรียงข้อมูล

STOP B STOP	- เป็นการ branch ไปที่ stop เพื่อเป็นการ loop เรื่อย ๆ เพื่อหยุดการทำงานในส่วนอื่น ๆ หรือ ว่าคือ หยุดการทำงานของโปรแกรม
PARTITION STMFD SP!, {R4-R8, LR}	- เก็บค่ารีจิสเตอร์ R4-R8 และ LR ลง Stack Memory
LSL R3, R2, #2 ADD R3, R0, R3	**R3 = Position Pivot** - เลื่อนบิตค่าของ high (R2) ไปทางซ้าย 2 บิต -> การคูณด้วย 4 เพื่อแปลงค่า index ของอาเรย์ให้เป็น byte offset ที่มีขนาด int = 4 ไบต์ และ ผลลัพธ์ถูกเก็บไว้ใน register R3 - นำค่า byte offset ใน R3 ไปบวกกับ base address ของอาเรย์ที่เก็บอยู่ใน R0 และเก็บค่าไว้ที่ register R0
LDR R4, [R3]	**R4 = pivot = array[high]** - โหลดค่าของ array[high] จากหน่วยความจำมาเก็บไว้ใน register R4 เพื่อใช้เป็นค่า pivot

SUB R5, R1, #1	**R5 = i = low - 1** - กำหนดค่า i = low - 1 โดยนำค่า low(R1) ลบด้วย 1 แล้วเก็บไว้ใน register R5
MOV R6, R1	**R6 = j = low (loop counter)** - กำหนดค่า j = low โดยคัดลอกค่า low จาก R1 ไปเก็บไว้ใน register R6 เพื่อใช้เป็นตัวแปรควบคุมลูป
PART_LOOP CMP R6, R2 BGE PART_END	**j < high** - เปรียบเทียบว่า j กับ high - ถ้า j >= high ให้ออกจากลูป

LSL R7, R6, #2
ADD R7, R7, R0
LDR R8, [R7]
	Load array[j]
	
	- เลื่อนบิตค่าของ j (R6) ไปทางซ้าย 2 บิต -> การคูณด้วย 4 เพื่อแปลงค่า index ของอาเรย์ให้เป็น byte offset ที่มีขนาด int = 4 ไบต์ และ ผลลัพธ์ถูกเก็บไว้ใน register R7
	
	- นำค่า byte offset ใน R7 ไปบวกกับ base address ของอาเรย์ที่เก็บอยู่ใน R0 เพื่อคำนวณ memory address ของ array[j] และนำไปเก็บไว้ที่ register R7
	
	- โหลดค่าของ array[j] จากหน่วยความจำมาเก็บไว้ใน register R8 เพื่อใช้ในการเปรียบเทียบกับค่า pivot ในขั้นตอน partition

CMP R8, R4 BGT PART_SKIP	**array[j] <= pivot** - เปรียบเทียบค่าของ array[j] (R8) กับค่า pivot (R4) - ถ้า array[j] > pivot ให้ข้ามขั้นตอนการ swap และไป PART_SKIP เลย

ADD R5, R5, #1	กรณี array[j] <= pivot - เพิ่มค่า i (R5)ขึ้นไป 1 เพื่อเลื่อนค่าไปยัง index ตำแหน่งถัดไปใน array และเก็บค่าไว้ที่ register R5

LSL R7, R5, #2
ADD R7, R7, R0
 	
LSL R8, R6, #2
ADD R8, R8, R0
 	
LDR R3, [R7]
LDR R12, [R8]
 	
STR R12, [R7]
STR R3, [R8]
	โหลดค่า **array[i]**
	
	- เลื่อนบิตค่าของ i (R5) ไปทางซ้าย 2 บิต -> การคูณด้วย 4 เพื่อแปลงค่า index ของอาเรย์ให้เป็น byte offset ที่มีขนาด int = 4 ไบต์ และ ผลลัพธ์ถูกเก็บไว้ใน register R7
	
	- นำค่า byte offset ใน R7 ไปบวกกับ base address ของอาเรย์ที่เก็บอยู่ใน R0 เพื่อคำนวณ memory address ของ array[i] และนำไปเก็บไว้ที่ register R7
	
	โหลดค่า **array[j]**
	
	- เลื่อนบิตค่าของ j (R6) ไปทางซ้าย 2 บิต -> การคูณด้วย 4 เพื่อแปลงค่า index ของอาเรย์ให้เป็น byte offset ที่มีขนาด int = 4 ไบต์ และ ผลลัพธ์ถูกเก็บไว้ใน register R8
	
	- นำค่า byte offset ใน R8 ไปบวกกับ base address ของอาเรย์ที่เก็บอยู่ใน R0 เพื่อคำนวณ memory address ของ array[j] และนำไปเก็บไว้ที่ register R8
	
	Swap
	- โหลดค่าของ array[i] จากหน่วยความจำมาเก็บไว้ใน register R3
	
	- โหลดค่าของ array[j] จากหน่วยความจำมาเก็บไว้ใน register R12

	- นำค่า array[j] ไปเก็บไว้ที่ตำแหน่ง array[i] - นำค่า array[i] เดิมไปเก็บไว้ที่ตำแหน่ง array[j] *เป็นการ Swapค่า ระหว่าง array[i] และ array[j]
PART_SKIP ADD R6, R6, #1 B PART_LOOP	- บวกค่า j เพิ่ม 1 - เป็นขั้นตอนจากการทำงาน BGT ถ้าหาก array[j] < pivot จะเข้ามาทำงานใน PART_SKIP หลังจากนั้นก็จะกลับไปที่ PART_LOOP

PART_END
ADD R5, R5, #1
 	
LSL R7, R5, #2
ADD R7, R7, R0
 	
LSL R8, R2, #2
	โหลดค่า **array[i + 1]**
	
	- บวกค่า i + 1
	- เลื่อนบิตค่าของ i + 1 (R5) ไปทางซ้าย 2 บิต -> การคูณด้วย 4 เพื่อแปลงค่า index ของอาเรย์ให้เป็น byte offset ที่มีขนาด int = 4 ไบต์ และ ผลลัพธ์ถูกเก็บไว้ใน register R7
	
	- นำค่า byte offset ใน R7 ไปบวกกับ base address ของอาเรย์ที่เก็บอยู่ใน R0 เพื่อคำนวณ memory address ของ array[i + 1] และนำไปเก็บไว้ที่ register R7
	
	โหลดค่า **array[high]**
	- เลื่อนบิตค่าของ high (R2) ไปทางซ้าย 2 บิต -> การคูณด้วย 4 เพื่อแปลงค่า index ของอาเรย์ให้เป็น byte offset ที่มีขนาด int = 4 ไบต์ และ ผลลัพธ์ถูกเก็บไว้ใน register R8
	- นำค่า byte offset ใน R8 ไปบวกกับ base address ของ

ADD R8, R8, R0
 	
LDR R3, [R7]
LDR R12, [R8]
 	
STR R12, [R7]
STR R3, [R8]
	อาเรย์ที่เก็บอยู่ใน R0 เพื่อคำนวณ memory address ของ array[high] และนำไปเก็บไว้ที่ register R8
	
	Swap
	- โหลดค่าของ array[i] จากหน่วยความจำมาเก็บไว้ใน register R3
	
	- โหลดค่าของ array[j] จากหน่วยความจำมาเก็บไว้ใน register R12
	
	- นำค่า array[high] ไปเก็บไว้ที่ตำแหน่ง array[i + 1]
	
	- นำค่า array[i + 1] เดิมไปเก็บไว้ที่ตำแหน่ง array[high]
	
	*เป็นการ Swap ค่า ระหว่าง array[i + 1] และ array[high]

MOV R0, R5 LDMFD SP!, {R4-R8, PC}	- คัดลอกค่าของตัวแปร i+1 ซึ่งเก็บอยู่ใน R5 มาใส่ไว้ใน register R0 เพื่อใช้เป็น ค่าผลลัพธ์ (pivot index) ที่จะส่งกลับไปยังฟังก์ชัน QUICKSORT - โหลดค่าจาก Stack Memory กลับมาใส่ใน register R4-R8 และ PC เพื่อคืนสถานะของโปรแกรมก่อนเข้าสู่ฟังก์ชัน PARTITION โดยการโหลดค่า PC จะทำให้โปรแกรมกลับไปทำงานต่อยังคำสั่งถัดจากจุดที่เรียกฟังก์ชัน PARTITION
QUICKSORT STMFD SP!, {R4-R6, LR}	- เก็บค่ารีจิสเตอร์ R4-R6 (base address ของ array, low และ high) และ LR ลง Stack Memory เพื่อรักษาค่าพารามิเตอร์ของฟังก์ชันและตำแหน่ง return

	ในกรณีที่มีการเรียกฟังก์ชันแบบ recursive
CMP R1, R2 BGE QS_END	**low < high** - เปรียบเทียบค่า low (R1) กับ high (R2) - หากค่า low >= high ให้จบการทำงานและข้ามไปที่พาร์ท QS_END เลย

MOV R4, R0
MOV R5, R1
MOV R6, R2
	Backup value
	
	- คัดลอกค่า base address ของอาเรย์จาก R0 ไปเก็บไว้ใน register R4
	
	- คัดลอกค่า low จาก R1 ไปเก็บไว้ใน register R5
	
	- คัดลอกค่า high จาก R2 ไปเก็บไว้ใน register R6
	* เพื่อสำรองค่าไว้ใช้งานภายหลัง เนื่องจาก R0-R2 จะถูกใช้เป็นพารามิเตอร์ในการเรียกฟังก์ชันอื่น

BL PARTITION	**Call Function** - Call function ของ PARTITION เพื่อทำการแบ่งอาเรย์ตามค่า pivot ซึ่งฟังก์ชัน PARTITION จะจัดตำแหน่งข้อมูลและส่งค่าตำแหน่งของ pivot กลับมาทาง register R0
MOV R3, R0	- เก็บค่าตำแหน่งของ pivot ที่ได้จากฟังก์ชัน PARTITION จาก R0 ไปยัง register R3

MOV R0, R4
MOV R1, R5
SUB R2, R3, #1
BL QUICKSORT
	quicksort(array, low, pivot_index - 1)
	- นำ base address ของอาเรย์ (R4) กลับมาใส่ใน register R0

	- คัดลอกค่า low จาก R5 ไปเก็บไว้ใน register R1 - คำนวณค่า high = pivot_index - 1 โดยนำค่าตำแหน่งของ pivot (R3) มาลบด้วย 1 และเก็บผลลัพธ์ไว้ใน register R2 - เรียกฟังก์ชัน QUICKSORT เพื่อทำการเรียงข้อมูลในอาเรย์ย่อยฝั่งซ้ายซึ่งเป็นค่าที่มีตำแหน่งน้อยกว่าตำแหน่งของ pivot

`MOV R0, R4`
`ADD R1, R3, #1`
`MOV R2, R6`
`BL QUICKSORT`
	quicksort(array, pivot_index + 1, high)
	
	- นำ base address ของอาเรย์ (R4) กลับมาใส่ใน register R0
	
	- คำนวณค่า
	low = pivot_index + 1
	โดยนำค่าตำแหน่งของ pivot (R3) มาบวกด้วย 1
	และเก็บผลลัพธ์ไว้ใน register R2
	
	- คัดลอกค่า high จาก R6 ไปเก็บไว้ใน register R2
	
	- เรียกฟังก์ชัน QUICKSORT
	เพื่อทำการเรียงข้อมูลในอาเรย์ย่อยฝั่งขวาซึ่งเป็นค่าที่มีตำแหน่งมากกว่าตำแหน่งของ pivot

QS_END
`LDMFD SP!, {R4-R6, PC}`
`END`
	- โหลดค่าจาก Stack Memory กลับมาใส่ใน register R4-R6 และ PC
	เพื่อคืนสถานะของโปรแกรมก่อนเข้าสู่ฟังก์ชัน QUICKSORT
	โดยการโหลดค่า PC จะทำให้โปรแกรมกลับไปทำงานต่อยังคำสั่งถัด

	จากจุดที่เรียกฟังก์ชัน QUICKSORT - จบการทำงานของโปรแกรม

Output result:

<table>
<tr><th>Output</th><th>Explanation</th></tr>
<tr>
<td>

Register:

Registers	Memory	Symbols
R0	512	
R1	8	
R2	8	
R3	7	
R4	0	
R5	0	
R6	0	
R7	0	
R8	0	
R9	0	
R10	0	
R11	0	
R12	11	
R13	-16777216	
R14	224	
R15	16	

</td>
<td>

Registers output นี้ คือค่าของ register หลังจากโปรแกรมทำงานเสร็จแล้วซึ่งเกิดจากการทำงานของ Quicksort

- R0 เป็นค่าที่ถูกใช้เพื่อเก็บ base address โดยมีค่าเท่ากับ 512 ซึ่งเป็นค่าที่ไม่ได้ restore จึงค้างอยู่ใน register หลังจบการทำงานของโปรแกรม
- R1 และ R2 มีค่าเป็น 8 เนื่องจากในขั้นตอนสุดท้ายของการเรียก Quicksort ค่า index ซ้าย (low) และขวา (high) มีค่าเท่ากัน ตรงกับเงื่อนไขการหยุด recursion ส่งผลให้ค่าดังกล่าวค้างอยู่ใน register
- R3 ที่มีค่าเป็น 7 เกิดจากการเก็บตำแหน่งของ pivot หลังการทำ partition รอบสุดท้าย ซึ่งเป็นตำแหน่งที่ pivot ถูกจัดเรียงเรียบร้อยแล้ว ในส่วนของ R12 มีค่าเป็น 11 เนื่องจาก register นี้ถูกใช้เป็นตัวแปรชั่วคราวในการโหลดและสลับข้อมูลใน array (swap) และไม่ได้ถูกเคลียร์ค่าหลังใช้งาน
- R4-R11 ไม่ปรากฏการเปลี่ยนแปลงในช่วงท้ายของโปรแกรมหรือถูกบันทึกและคืนค่าระหว่างการเรียกฟังก์ชัน จึงมีค่าเป็น 0 ใน output
- R13 ซึ่งเป็น Stack Pointer แสดงค่าที่ตัว simulator กำหนดไว้เริ่มต้น
- R14 เป็น Link Register ที่เก็บตำแหน่งคำสั่งถัดไปหลังการเรียกฟังก์ชันครั้งล่าสุด
- R15 มีค่าเป็น 16 เนื่องจากโปรแกรมเข้าสู่คำสั่ง B STOP ทำให้ตัวประมวลผลวนอยู่ที่ตำแหน่งเดิม ส่งผลให้ค่า Program Counter ค้างอยู่ใน register

</td>
</tr>
</table>

Output	Explanation

Memory:

Registers | Memory | Symbols

Enable Byte View

Enable Reverse Direction

Symbol	Address	Value
Data	0×200	1
	0×204	5
	0×208	6
	0×20C	7
	0×210	8
	0×214	8
	0×218	10
	0×21C	11
	0×220	21
Symbol	**Address**	**Value**
	0×FEFFFFA8	512
	0×FEFFFFAC	2
	0×FEFFFFB0	512
	0×FEFFFFB4	2
	0×FEFFFFB8	512
	0×FEFFFFBC	6
	0×FEFFFFC0	512
	0×FEFFFFC4	6
	0×FEFFFFC8	8
	0×FEFFFFCC	224
	0×FEFFFFD0	512
	0×FEFFFFD4	4
	0×FEFFFFD8	8
	0×FEFFFFDC	224
	0×FEFFFFE0	512
	0×FEFFFFE4	0
	0×FEFFFFE8	8
	0×FEFFFFEC	224
	0×FEFFFFF0	0
	0×FEFFFFF4	0
	0×FEFFFFF8	0
	0×FEFFFFFC	16

Uninitialized memory is zeroed

N 0 Z 1 C 1 V 0

- Memory output ส่วนแรกเป็นพื้นที่ข้อมูล Data ซึ่งถูกกำหนดไว้ด้วยคำสั่ง DCD โดยเริ่มที่ address 0x200 และเพิ่มทีละ 4 ไบต์ตามขนาดของข้อมูลแบบ word ค่าใน memory แสดงให้เห็นว่าข้อมูลถูกเรียงลำดับแล้วเป็น 1, 5, 6, 7, 8, 8, 10, 11, 21 ซึ่งเป็นผลลัพธ์จากการทำงานของ Quicksort โดยโปรแกรมได้ทำการ partition และ swap ค่าภายใน array หลายครั้งจนข้อมูลอยู่ในลำดับจากน้อยไปมาก ทำให้ค่าที่แสดงใน memory แตกต่างจากค่าตั้งต้น แต่สอดคล้องกับผลลัพธ์ของการเรียงลำดับที่ถูกต้อง

- ส่วน memory ช่วง address 0xFFFFFFxx เป็นพื้นที่ของ stack ซึ่งถูกใช้งานระหว่างการเรียกฟังก์ชัน QUICKSORT แบบ recursive โดยคำสั่ง STMFD และ LDMFD ใช้ในการเก็บและคืนค่า register ต่าง ๆ เช่น ค่า base address ของ Data, ค่า index ซ้ายและขวา รวมถึงค่า return address ในแต่ละระดับของการเรียกฟังก์ชัน ทำให้เห็นค่าซ้ำ ๆ เช่น 512, 8, 6 และ 224 ปรากฏอยู่หลายตำแหน่งใน stack ค่า 224 คือ return address ที่ถูกเก็บจาก LR ขณะที่ค่า 512 คือ base address ของข้อมูลซึ่งถูก push ลง stack เพื่อป้องกันการสูญหายระหว่างการเรียกฟังก์ชันซ้อนกันหลายชั้น ส่วนตำแหน่งที่มีค่าเป็น 0 แสดงถึงพื้นที่ stack ที่ยังไม่ได้ถูกใช้งานหรือถูกคืนค่าหลังจากจบการทำงานของฟังก์ชัน

Output	Explanation
Symbols: Registers | Memory | Symbols Code Symbol | Value MAIN | 0 STOP | 16 PARTITION | 20 PART_LOOP | 44 PART_SKIP | 108 PART_END | 116 QUICKSORT | 160 QS_END | 224 Data Symbol | Value Data | 512	• ตารางส่วน Code Symbol แสดงตำแหน่ง address ของ label ต่าง ๆ ที่ถูกประกาศไว้ใน source code ซึ่ง assembler จะทำการแปลง label เหล่านี้ให้เป็นค่าตัวเลขที่แทนตำแหน่งคำสั่งจริงในหน่วยความจำ โดย MAIN มีค่าเป็น 0 เนื่องจากเป็นจุดเริ่มต้นของโปรแกรมและเป็นคำสั่งแรกที่ถูกจัดวางใน code section ถัดมา STOP มีค่าเป็น 16 เพราะอยู่ถัดจากคำสั่งใน MAIN ตามลำดับของโค้ด ส่วน PARTITION, PART_LOOP, PART_SKIP และ PART_END มีค่าเป็น 20, 44, 108 และ 116 ตามลำดับ ซึ่งสะท้อนตำแหน่งของคำสั่งภายในฟังก์ชัน partition โดยค่าที่เพิ่มขึ้นเกิดจากจำนวนคำสั่งก่อนหน้าแต่ละ label เมื่อแปลงเป็น address จริงแล้ว assembler จะนับเป็นหน่วยไบต์ ทำให้ค่าของแต่ละ symbol แตกต่างกันตามความยาวของโค้ดในแต่ละช่วง • สำหรับ QUICKSORT ซึ่งมีค่าเป็น 160 เป็นตำแหน่งเริ่มต้นของฟังก์ชัน quicksort ที่ถูกเรียกจาก MAIN และจากตัวมันเองแบบ recursive ส่วน QS_END มีค่าเป็น 224 ซึ่งเป็นตำแหน่งของคำสั่งสิ้นสุดฟังก์ชันเมื่อเข้าเงื่อนไข low >= high และค่านี้ยังสอดคล้องกับค่า 224 ที่ปรากฏใน register R14 (LR) และใน stack memory เนื่องจากเป็น return address ที่ถูกบันทึกไว้ระหว่างการเรียกฟังก์ชัน • ในส่วน Data Symbol ค่า Data = 512 แสดงถึง base address ของข้อมูลที่ถูกประกาศด้วย DCD ในโค้ด ซึ่งตรงกับค่าที่ปรากฏใน register R0 และถูก push ลง stack หลายครั้งระหว่างการทำงานของโปรแกรม ดังนั้น Symbols output จึงเป็นแผนที่ที่เชื่อมโยงโดยตรงระหว่าง label ในโค้ดกับตำแหน่งจริงในหน่วยความจำ ซึ่งอธิบายได้ว่าทำไมค่าบางตัว เช่น 16, 160 และ 224 จึงไปปรากฏซ้ำใน register และ memory output ของโปรแกรม